TALLER N° 2 DE PROGRAMACIÓN

Asignatura: Programación | Prof. Wilmer Varón

Proyecto: FIM-Secure (Monitor de Integridad de Archivos)

Estudiantes: Nelson Sánchez, Andrea Pernil, Pedro Diaz

Institución: Universidad Nacional de Las Ciencias

1. INTRODUCCIÓN Y PROPÓSITO

El objetivo del presente taller es diseñar y verificar la capa de consultas SQL del sistema **FIM-Secure**. Se evalúa el funcionamiento de las operaciones CRUD fundamentales para la gestión de usuarios, políticas de auditoría, líneas base de archivos y detección de alertas, garantizando al mismo tiempo que la implementación sea inmune a vulnerabilidades de **Inyección SQL (SQLi)**.

2. ESTADO DE SIMULACIÓN DE LA BASE DE DATOS

Para validar el comportamiento del motor relacional (SQLite / PostgreSQL), se simula la inserción de registros en las tablas creadas mediante el script DDL del Taller N° 1:

Tabla	Registro Simulado	Atributos Principales
USUARIO	`id: 1, 2`	`nelson_sec` (Admin), `pedro_diaz` (Analista)
POLITICA	`id: 1, 2`	Monitoreo Sistema Operativo (3600s), Monitoreo Servidores Web (1800s)
ARCHIVO	`id: 1, 2`	`/etc/passwd`, `/var/www/html/index.php` (Línea Base SHA-256)

3. CONSULTA SQL 1: RELACIÓN DE POLÍTICAS Y CREADORES

Obtiene las políticas activas de escaneo junto al nombre del usuario que las definió:

```
SELECT p.id_politica, p.nombre_politica, p.frecuencia_seg, u.username AS Creador FROM POLITICA p INNER JOIN USUARIO u ON p.id_usuario = u.id_usuario;
```

Resultado devuelto por la base de datos:

id_politica	nombre_politica	frecuencia_seg	Creador
1	Monitoreo Sistema Operativo	3600	nelson_sec
2	Monitoreo Servidores Web	1800	pedro_diaz

4. CONSULTA SQL 2: REPORTE FORENSE DE INCIDENTES DE SEGURIDAD

Correlaciona las discrepancias detectadas por el motor en C/Python en la tabla ALERTA con la ruta original registrada en la línea base de ARCHIVO:

```
SELECT al.id_alerta, ar.ruta_absoluta, al.tipo_evento, al.sha256_detectado, al.timesta
mp FROM ALERTA al INNER JOIN ARCHIVO ar ON al.id_archivo = ar.id_archivo;
```

Resultado devuelto por la base de datos:

id_alerta	ruta_absoluta	tipo_evento	sha256_detectado	timestamp
1	/var/www/html/index.php	MODIFIED	cf83e1357eefb8bd...	2026-10-29 14:22:10

5. COMPROBACIÓN DE INMUNIDAD CONTRA SQL INJECTION

Para evitar ataques de inyección SQL (donde un atacante ingresa valores como `' OR '1'='1` para eludir la autenticación o extraer datos no autorizados), el sistema implementa tres capas de protección estricta:

```
# Implementación Segura en Python (Prepared Statements) def autenticar_analista(conexi
on, username_ingresado, password_hash_ingresado): cursor = conexion.cursor() # Los caractere
s '?' separan la lógica SQL de los datos ingresados por el usuario consulta = "SELECT id_usu
ario, rol FROM USUARIO WHERE username = ? AND password_hash = ?" cursor.execute(consulta, (u
sername_ingresado, password_hash_ingresado)) return cursor.fetchone()
```

Mecanismos de Protección Activos:

- **Consultas Preparadas (Parameterized Queries):** Garantizan que el motor trate las entradas del usuario estrictamente como literales y nunca como código executable.
- **Restricciones de Dominio en DDL:** Los campos de tipo hash están definidos rigurosamente como `CHAR(64)`, impidiendo la entrada de cadenas de longitud arbitraria o comandos SQL adicionales.
- **Cláusulas CHECK:** Limitan los valores de campos sensibles como `rol ('Admin', 'Analista')` y `tipo_evento ('MODIFIED', 'DELETED', 'CREATED')`.